SAECOIN

DEVELOPMENT ROADMAP

Version 1.0

Project: SAECOIN
Ticker: SAC (Provisional)
Project Type: Independent Layer-1 Blockchain
Consensus: Proof of Work (PoW)
Transaction Model: UTXO
Development Environment: Windows 11, 16 GB RAM
Status: Planning

---

1. ROADMAP OBJECTIVE

This document defines the complete development roadmap for SAECOIN.

The project will be developed in structured phases, beginning with protocol research and specification finalization, followed by blockchain core development, testing, wallets, mining, explorer, testnet, security audits, and mainnet preparation.

The project must not skip critical technical or security stages.

The goal is to build a functional, independently verifiable, secure, and maintainable Layer-1 blockchain.

2. DEVELOPMENT PHILOSOPHY

SAECOIN will follow these principles:

1. Documentation before implementation.
2. Consensus rules before node development.
3. Small modules before complete integration.
4. Automated testing throughout development.
5. Local development before public testnet.
6. Security review before mainnet.
7. No premature token or coin launch.
8. No real-money activity on development networks.
9. Every major milestone requires verification.
10. Consensus changes require documented review and compatibility testing.

3. PROJECT DEVELOPMENT PHASES

The project is divided into 14 major phases.

PHASE 0 — DOCUMENTATION AND PLANNING

Objective: Establish the complete technical and product requirements before coding.

Documents:

- PRD.md
- AppFlow.md
- UI_Spec.md
- TRD.md
- Schema.md
- Blockchain_Architecture.md
- Consensus_Spec.md
- Security_Spec.md
- API_Spec.md
- Testing_Strategy.md
- Development_Roadmap.md

Additional documents still required:

- GitHub_Repository_Structure.md
- Coding_Standards.md
- Build_and_Release.md
- Network_Parameters.md
- Genesis_Block_Spec.md
- Wallet_Security_Design.md

Completion criteria:

- Required documents reviewed.
- Unresolved technical decisions listed.
- Proposed parameters clearly distinguished from finalized rules.
- No coding begins before the initial architecture review.

PHASE 1 — PROTOCOL RESEARCH AND FINALIZATION

Objective: Finalize the fundamental blockchain protocol decisions.

Tasks:

1. Select the proof-of-work algorithm.
2. Select the block hashing algorithm.
3. Select transaction signature scheme.
4. Define transaction serialization.

5. Define block serialization.
6. Define block header format.
7. Define transaction identifier calculation.
8. Define UTXO validation rules.
9. Define block size or weight limits.
10. Define maximum transaction size.
11. Define difficulty adjustment formula.
12. Define timestamp rules.
13. Define coinbase maturity.
14. Define transaction fee rules.
15. Define dust policy, if needed.
16. Define mempool policy.
17. Mathematically verify the supply schedule.
18. Define chain selection rules.
19. Define network message protocol.
20. Define genesis block parameters.

Important decisions:

The following values are provisional:

- Maximum supply: 21,000,000 SAC.
- Initial reward: 50 SAC.
- Block time: 10 minutes.
- Halving interval: 210,000 blocks.
- Premine: 0.
- Decimal precision: 8.

The protocol must not be considered finalized until these values and their implementation have been independently reviewed.

Completion criteria:

- Consensus specification finalized.
- Test vectors prepared.
- Genesis parameters documented.
- Protocol decisions reviewed.

PHASE 2 — DEVELOPMENT ENVIRONMENT AND GITHUB

Objective: Prepare a clean and reproducible development environment.

Tasks:

1. Install Rust toolchain.
2. Configure Git.

3. Create GitHub repository.
4. Establish project directory structure.
5. Configure automated formatting.
6. Configure automated testing.
7. Configure dependency auditing.
8. Establish branching strategy.
9. Create development documentation.
10. Verify the first minimal build.

Suggested repository name:

"saecoin"

Suggested top-level structure:

```
saecoin/
├── docs/
├── core/
├── node/
├── wallet/
├── miner/
├── explorer/
├── api/
├── android/
├── desktop/
├── tests/
├── scripts/
├── .github/
└── README.md
```

This is a provisional structure and may be adjusted after reviewing the Rust workspace architecture.

Completion criteria:

- Repository created.
- Development environment verified.
- Initial project builds.
- Documentation stored in the repository.

PHASE 3 — BLOCKCHAIN CORE

Objective: Implement the foundational blockchain logic.

Tasks:

1. Implement protocol data types.
2. Implement deterministic serialization.
3. Implement hashing interfaces.
4. Implement transaction structures.
5. Implement block structures.
6. Implement block headers.
7. Implement address and key interfaces.
8. Implement signature verification.
9. Implement UTXO state management.
10. Implement transaction validation.
11. Implement block validation.
12. Implement subsidy and fee calculation.
13. Implement consensus validation interfaces.

Testing:

- Unit tests.
- Serialization test vectors.
- Transaction validation tests.
- Block validation tests.
- Overflow tests.
- Invalid input tests.

Completion criteria:

- Core library builds.
- Consensus-related tests pass.
- Independent test vectors produce identical results.

PHASE 4 — BLOCKCHAIN DATABASE

Objective: Store and recover blockchain state reliably.

Tasks:

1. Configure the selected database.
2. Implement block storage.
3. Implement transaction indexing.
4. Implement UTXO storage.
5. Implement block index.
6. Implement chain metadata.
7. Implement undo data.
8. Implement atomic state updates.
9. Implement database recovery.
10. Implement reindexing.

Testing:

- Database restart.
- Crash recovery.
- Reindexing.
- Corruption handling.
- Chain reorganization recovery.

Completion criteria:

- Blocks persist correctly.
- UTXO state remains consistent.
- Database recovery tests pass.

PHASE 5 — FULL NODE

Objective: Build the first complete SAECOIN full node.

Tasks:

1. Node initialization.
2. Configuration management.
3. Genesis block loading.
4. Blockchain validation.
5. Transaction validation.
6. Mempool management.
7. Block acceptance.
8. Chain selection.
9. Chain reorganization.
10. Node status reporting.
11. Graceful shutdown.
12. Restart recovery.

Completion criteria:

- Node starts successfully.
- Node validates blocks independently.
- Node maintains a consistent blockchain.
- Node survives restart and recovery tests.

PHASE 6 — P2P NETWORK

Objective: Connect multiple full nodes into an independent network.

Tasks:

1. Define peer protocol.
2. Implement peer connections.
3. Implement peer discovery.
4. Implement handshake.
5. Implement message framing.
6. Implement block propagation.
7. Implement transaction propagation.
8. Implement chain synchronization.
9. Implement peer scoring.
10. Implement timeout handling.
11. Implement connection limits.
12. Implement network recovery.

Testing:

- Multiple nodes.
- Disconnected nodes.
- Malicious peers.
- Network partitions.
- Reconnection.
- Synchronization.

Completion criteria:

- Multiple independent nodes communicate.
- Blocks propagate correctly.
- Nodes synchronize the same valid chain.

PHASE 7 — MINING SYSTEM

Objective: Enable proof-of-work block mining.

7.1 CPU Mining

Tasks:

- Mining job creation.
- Nonce search.
- Hash calculation.
- Target comparison.
- Valid solution submission.
- Hashrate reporting.
- Mining configuration.

7.2 GPU Mining

Tasks:

- GPU device detection.
- GPU computation implementation.
- Mining job distribution.
- Nonce search.
- Solution verification.
- GPU performance reporting.
- Error handling.

7.3 Full Node Integration

The full node must independently verify every submitted block.

CPU and GPU miners must follow identical consensus rules.

Completion criteria:

- CPU mining works.
- GPU mining works on supported hardware.
- Full node accepts valid blocks.
- Full node rejects invalid solutions.

PHASE 8 — WALLET CORE

Objective: Develop the shared wallet functionality.

Tasks:

1. Secure key generation.
2. Address generation.
3. Wallet encryption.
4. Seed backup.
5. Wallet restoration.
6. UTXO discovery.
7. Balance calculation.
8. Transaction construction.
9. Local transaction signing.
10. Transaction broadcasting.
11. Transaction history.
12. Confirmation tracking.

Security requirements:

- Private keys remain under user control.
- Signing occurs locally.
- Remote APIs must not receive seed phrases.
- Backup and recovery must be tested.

Completion criteria:

- Wallet core passes security tests.
- Transactions can be created and signed.
- Wallet restoration works correctly.

PHASE 9 — ANDROID WALLET

Objective: Build the SAECOIN Android application.

Technology:

- Kotlin.
- Jetpack Compose.
- Shared Rust wallet core where technically practical.

Features:

1. Create wallet.
2. Restore wallet.
3. Secure wallet access.
4. Display SAC balance.
5. Receive SAC.
6. Send SAC.
7. QR code support.
8. Transaction history.
9. Confirmation status.
10. Network selection.
11. Wallet backup.
12. Node connection status.

Testing:

- Android compatibility.
- Storage security.
- Backup and restore.
- Transaction signing.
- Network interruption.
- Application restart.

Completion criteria:

- Android wallet operates against the private testnet.
- Wallet security and recovery tests pass.

PHASE 10 — WINDOWS DESKTOP WALLET

Objective: Build the Windows wallet application.

Technology:

- Tauri.
- Rust.
- TypeScript.

Features:

- Wallet creation.
- Wallet restoration.
- SAC balance.
- Receive SAC.
- Send SAC.
- QR support.
- Transaction history.
- Wallet backup.
- Node connection.
- Network selection.

Completion criteria:

- Windows wallet operates against the private testnet.
- Transaction and recovery tests pass.

PHASE 11 — API AND BLOCK EXPLORER

Objective: Make blockchain information accessible through documented APIs and an explorer.

API

Tasks:

- Node status.
- Blockchain information.
- Block lookup.

- Transaction lookup.
- Transaction broadcast.
- Network information.
- Mining status.
- API security.
- Rate limiting.

Explorer

Tasks:

- Block list.
- Block details.
- Transaction details.
- Address information.
- Supply information.
- Network statistics.
- Search functionality.
- Reorganization handling.

Completion criteria:

- API responses match node data.
- Explorer indexes the active chain correctly.
- Reindexing and recovery work.

PHASE 12 — PRIVATE TESTNET

Objective: Test the complete blockchain in a controlled environment.

Tasks:

1. Generate testnet genesis block.
2. Configure testnet parameters.
3. Start multiple independent nodes.
4. Connect nodes through P2P.
5. Start CPU miners.
6. Test GPU miners.
7. Create test wallets.
8. Transfer test SAC.
9. Test transaction confirmations.
10. Test network interruption.
11. Test chain reorganizations.
12. Test database recovery.
13. Test wallet backup and restore.
14. Test malicious network messages.

15. Benchmark resource consumption.

The testnet must use a completely separate identity from mainnet.

Completion criteria:

- All major components work together.
- Critical bugs are resolved.
- Testnet results are documented.
- Independent nodes maintain consensus.

PHASE 13 — SECURITY AUDIT AND MAINNET PREPARATION

Objective: Prepare the project for a safe public release.

Tasks:

1. Final consensus review.
2. Supply schedule verification.
3. Cryptographic review.
4. Wallet security review.
5. P2P security review.
6. Database recovery review.
7. API security review.
8. Dependency audit.
9. Fuzz testing.
10. Independent security audit.
11. Release build verification.
12. Genesis block verification.
13. Mainnet configuration review.
14. Backup and recovery plan.
15. Incident response plan.
16. Mainnet launch checklist.

Completion criteria:

- No unresolved critical security findings.
- Consensus rules are frozen for launch.
- Independent audit findings are addressed.
- Mainnet configuration is verified.
- Release approval is documented.

PHASE 14 — MAINNET LAUNCH AND MAINTENANCE

Objective: Launch and maintain the SAECOIN mainnet.

Tasks:

1. Publish official node software.
2. Publish wallet applications.
3. Publish mining software.
4. Publish network documentation.
5. Publish genesis information.
6. Publish consensus test vectors.
7. Activate mainnet.
8. Monitor node health.
9. Monitor network synchronization.
10. Monitor mining activity.
11. Monitor security alerts.
12. Release maintenance updates.
13. Publish transparent release notes.

Mainnet must not be launched simply to meet a deadline.

Security and consensus correctness take priority over launch speed.

---

4. DEVELOPMENT PRIORITY

Recommended order:

Priority| Component| Importance
1| Protocol finalization| Critical
2| Blockchain core| Critical
3| Consensus validation| Critical
4| Database| Critical
5| Full node| Critical
6| P2P networking| Critical
7| Mining| High
8| Wallet core| High
9| Android wallet| High
10| Windows wallet| High
11| API| Medium
12| Explorer| Medium
13| Private testnet| Critical release gate
14| Security audit| Critical release gate
15| Mainnet| Final milestone

The exact order of some implementation tasks may change based on technical dependencies.

5. DEVELOPMENT APPROACH

Development will proceed module by module.

For each module:

1. Review requirements.
2. Define interfaces.
3. Implement the smallest working version.
4. Compile.
5. Run unit tests.
6. Run integration tests.
7. Review security.
8. Update documentation.
9. Commit to GitHub.
10. Approve the module before moving forward.

AI-assisted development may be used, but generated code must be reviewed, tested, and understood before integration.

AI-generated code must never be trusted as correct merely because it compiles.

6. VERSION CONTROL STRATEGY

Suggested branches:

- "main" — stable approved releases.
- "develop" — ongoing development.
- "feature/*" — new features.
- "bugfix/*" — bug fixes.
- "release/*" — release preparation.

Every major milestone should have a Git tag.

Example:

- "v0.1.0" — Initial core.
- "v0.2.0" — Basic full node.
- "v0.3.0" — P2P integration.
- "v0.4.0" — Mining.
- "v0.5.0" — Wallet core.
- "v0.6.0" — Android and desktop wallets.
- "v0.7.0" — Explorer and API.
- "v0.8.0" — Private testnet.
- "v0.9.0" — Release candidate.
- "v1.0.0" — Mainnet release, only after approval.

These version numbers are planning labels, not fixed deadlines or promises.

7. PROJECT RESOURCE MANAGEMENT

Initial development hardware:

- Operating system: Windows 11.
- RAM: 16 GB.

Development should be optimized for this environment.

Recommendations:

- Build and test one major component at a time.
- Avoid unnecessary background services.
- Keep local development databases manageable.
- Use incremental builds.
- Benchmark before selecting production hardware.
- Use external infrastructure only when required.

A larger server may be introduced later for public testnet or mainnet infrastructure.

8. RISK MANAGEMENT

Major project risks:

Risk	Mitigation
Consensus bug	Formal specification and test vectors
Supply calculation error	Independent mathematical verification
Wallet key exposure	Local signing and security review
Network attacks	P2P validation and rate limiting
Database corruption	Atomic writes and recovery testing
GPU incompatibility	Hardware compatibility testing
Poor performance	Benchmarking and profiling
AI-generated code defects	Code review and automated tests
Scope expansion	Strict phase-based development
Premature mainnet launch	Mandatory release gates

9. SUCCESS CRITERIA

SAECOIN development will be considered successful when:

1. It operates as an independent blockchain.
2. Its full nodes independently validate consensus.
3. Its PoW system works correctly.
4. CPU and GPU miners can produce valid solutions.
5. Its UTXO transaction system operates reliably.
6. Its Android wallet works.
7. Its Windows wallet works.

8. Its P2P network synchronizes correctly.
9. Its explorer displays accurate blockchain information.
10. Its supply rules are mathematically verified and enforced.
11. Its security and recovery requirements are satisfied.
12. Its mainnet passes all release gates.

END OF DOCUMENT